

70. CSS Animations

CSS animations se bina JS ke simple animations ban jaati hain. JS se inhe control karke aur behtar bhi kar sakte hain (thode code me).

CSS Transitions

Idea simple: ek property batao aur uske badalne par kaise animate ho. Property badlo, **browser khud smooth transition paint karta hai.**

CSS

```
.animated {  
  transition-property: background-color;  
  transition-duration: 3s;  
}
```

Ab `.animated` element ka `background-color` badalne par 3 second me animate hoga.

js

```
color.onclick = function() { this.style.backgroundColor = 'red'; };
```

4 properties:

- `transition-property`
- `transition-duration`
- `transition-timing-function`
- `transition-delay`

Shorthand `transition` (order: `property duration timing-function delay`), kai properties ek saath:

CSS

```
#growing { transition: font-size 3s, color 2s; }
```

`transition-property`

Animate hone wali properties ki list (`left`, `margin-left`, `height`, `color`) ya `all`. Kuch properties animate nahi hoti par zyaadatar hoti hain.

`transition-duration`

Kitni der. CSS time format me: `s` ya `ms`.

transition-delay

Animation shuru hone se **pehle** ka delay. `delay: 1s`, `duration: 2s` ho to property badalne ke 1 sec baad shuru, total 2 sec. **Negative delay** bhi ho sakta hai: animation turant dikhta hai par beech se shuru (delay `-1s`, duration `2s`) ho to aadhe rasta se shuru, total 1 sec).

js

```
stripe.style.transitionDelay = '--' + sec + 's'; // JS se
```

transition-timing-function

Animation ki speed timeline par kaise baante: pehle dheere phir tez, ya ulta. 2 tarah ki values: **Bezier curve** ya **steps**.

Bezier curve

`cubic-bezier(x2, y2, x3, y3)`: 4 control points, jisme pehla `(0,0)` aur aakhri `(1,1)` fixed. Sirf 2nd aur 3rd dene hain. Beech ke points ka `x` `0..1` me hona chahiye, `y` kuch bhi.

- `x` axis: **time** (0 = shuru, 1 = `transition-duration` ka end)
- `y` axis: **progress** (0 = shuruaati value, 1 = final value)

`linear` = `cubic-bezier(0, 0, 1, 1)`, seedhi line, constant speed.

Train ko dheere hote dikhana: `cubic-bezier(0.0, 0.5, 0.5, 1.0)` (shuru me tez, phir dheere).

Built-in names: `linear`, `ease`, `ease-in`, `ease-out`, `ease-in-out`.

Name

cubic-bezier

<code>ease</code> (default)	<code>(0.25, 0.1, 0.25, 1.0)</code>
<code>ease-in</code>	<code>(0.42, 0, 1.0, 1.0)</code>
<code>ease-out</code>	<code>(0, 0, 0.58, 1.0)</code>
<code>ease-in-out</code>	<code>(0.42, 0, 0.58, 1.0)</code>

Bezier curve animation ko apne range se bahar bhi le ja sakti hai. `y` negative ya `1` se zyada ho to property shuru se pehle ya end se aage bhi jaati hai:

css

```
.train { left: 100px; transition: left 5s cubic-bezier(.5, -1, .5, 2); }  
/* left 100px se 400px: pehle peeche jaata hai (<100px), phir aage (>400px), ph
```

Curve banane ke tools: <https://cubic-bezier.com>, aur browser DevTools (Styles panel me `cubic-bezier` ke aage icon click karo).

Steps

`steps(number of steps[, start/end])`: transition ko **discrete steps** me todta hai. Example: digits ka timer. `#stripe` ko `#digit` ke window ke bahar `overflow: hidden` se chhupao aur 9 steps me shift karo:

CSS

```
#stripe.animate {  
  transform: translate(-90%);  
  transition: transform 9s steps(9, start);  
}
```

- Pehla argument: steps ki ginti (transform 9 hisson me, time bhi 9 hisson me, yani har second ek digit)
- Doosra: `start` ya `end`
 - `start`: animation ke shuru me pehla step **turant**
 - `end`: har step second ke **end** me (pehle second kuch nahi badalta)

Shorthands: `step-start` = `steps(1, start)`, `step-end` = `steps(1, end)` (asli animation nahi, single-step change, kam use).

`transitionend` event

CSS animation khatam hone par chalta hai. Animation ke baad kuch karne ya animations jodne ke kaam aata hai.

js

```
boat.addEventListener('transitionend', function() {  
  times++;  
  go();      // agla animation  
});
```

Event properties:

- `event.propertyName`: jo property animate ho ke khatam hui (kai ek saath animate ho to kaam ka)
- `event.elapsedTime`: kitne seconds lage (`transition-delay` ke bina)

Keyframes

Kai simple animations jodne ke liye `@keyframes`. Naam aur rules (kya, kab, kahan) batao, phir `animation` property se element par lagao:

```
css

@keyframes go-left-right {
  from { left: 0px; }
  to   { left: calc(100% - 50px); }
}

.progress {
  animation: go-left-right 3s infinite alternate;
  /* naam, duration, infinite baar, har baar direction ulta (alternate) */
  position: relative;
}
```

Zyaada baar zarurat nahi padti jab tak sab kuch lagatar chalta na ho.

Performance

Zyaadatar CSS properties (numeric) animate hoti hain, par sab **barabar smooth** nahi hoti, kyunki alag properties ko badalne ki cost alag hai. Style badalne par browser 3 steps karta hai:

1. **Layout:** har element ki geometry aur position phir se
2. **Paint:** dikhna kaisa hai (background, colors)
3. **Composite:** final pixels screen par, CSS transforms lagana

Animation me ye har frame hota hai. Jo properties geometry nahi badalti (jaise `color`) wo Layout **skip** kar deti hain. Kuch to seedhe Composite tak jaati hain. Properties kaun sa stage trigger karti hain: <https://csstriggers.com>.

Bahut elements aur complex layout wale pages par delay dikhta hai ("jittery" animation).

`transform` behtar hai:

- Element box par poore ka asar (rotate, flip, stretch, shift), padosi elements par nahi
- Browser ise Layout aur Paint ke **upar**, Composite stage me lagata hai, isliye **Layout aur Paint trigger nahi** hote
- Graphics accelerator (GPU) use hota hai, isliye bahut efficient

`left/margin-left` ki jagah `transform: translateX(...)`, size badhane ke liye `transform: scale(...)`.

`opacity` bhi Layout trigger nahi karta (Mozilla Gecko me Paint bhi nahi). Show/hide, fade-in/out ke liye.

`transform` + `opacity` milkar zyaadatar zarurat pure kar dete hain:

CSS

```
#boat { transition: transform 2s ease-in-out, opacity 2s ease-in-out; }  
.move { transform: translateX(300px); opacity: 0; }
```

js

```
boat.onclick = () => boat.classList.add('move');
```

Aur complex `@keyframes` example: `0%` me `translateY(-60px) rotateX(0.7turn)` + `opacity: 0`, `50%` me `none` + `opacity: 1`, `100%` me `translateX(230px) rotateZ(90deg) scale(0.5)` + `opacity: 0`.

Summary

CSS animations ek ya kai properties ke smooth (ya step-by-step) badlav ke liye hain. Zyaadatar animation tasks ke liye achhi hain.

Fayde: simple cheezein simple tarike se, CPU par tez aur halki. **Kamiyan:** JS animations zyaada flexible hain (koi bhi logic, jaise element ka "explosion", naye elements banana). Sirf property changes nahi.

Real projects me `font-size`, `left`, `width`, `height` ki jagah `transform: scale()` aur `transform: translate()` use karo (behtar performance). `transitionend` se JS ko animation ke baad chalane me integrate kar sakte hain. Agle chapter me JS animations.

Tasks (short)

- **Plane grow (CSS):** `#flyjet { transition: all 3s; }` aur JS `.growing` class lagaye (`width: 400px; height: 240px`). `transitionend` **do baar** (width aur height dono ke liye) chalta hai, to check lagao warna "Done!" 2 baar dikhega.
- **Plane "jump out":** bezier me `y > 1`, jaise `cubic-bezier(0.25, 1.5, 0.75, 1.5)`.
- **Animated circle:** `showCircle(cx, cy, radius)` circle ko `transition` se grow karta hai.
- **Callback ke saath:** `showCircle(cx, cy, radius, callback)`; animation khatam hone par `transitionend` `callback(div)` chalao.